Investigating Overlapped Strategies to Solve Overlapping Problems in a Cooperative Co-evolutionary Framework

Julien Blanchard[1(✉)], Charlotte Beauthier[2], and Timoteo Carletti[1]

[1] Department of Mathematics and naXys institute, University of Namur, Namur, Belgium
{julien.blanchard,timoteo.carletti}@unamur.be
[2] Cenaero Research Center, Gosselies, Belgium
charlotte.beauthier@cenaero.be

Abstract. Cooperative co-evolution is recognized as an effective approach for solving large-scale optimization problems. It breaks down the problem dimensionality by splitting a large-scale problem into ones focusing on a smaller number of variables. This approach is successful when the studied problem is decomposable. However, many practical optimization problems can not be split into disjoint components. Most of them can be seen as interconnected components that share some variables with other ones. Such problems composed of parts that overlap each other are called overlapping problems. This paper proposes a modified cooperative co-evolutionary framework allowing to deal with non-disjoint subproblems in order to decompose and optimize overlapping problems efficiently. The proposed algorithm performs a new decomposition based on differential grouping to detect overlapping variables. A new cooperation strategy is also introduced to manage variables shared among several components. The performance of the new overlapped framework is assessed on large-scale overlapping benchmark problems derived from the CEC'2013 benchmark suite and compared with a state-of-the-art non-overlapped framework designed to tackle overlapping problems.

Keywords: Large-scale global optimization · Evolutionary algorithms · Cooperative co-evolution · Overlapping problem

1 Introduction

Nowadays, many real-world optimization problems arising in engineering and sciences deal with a large number of variables [7]. They present challenging

The present research benefited from computational resources made available on the Tier-1 supercomputer of the Fédération Wallonie-Bruxelles, infrastructure funded by the Walloon Region under the grant agreement n°1117545.

B. Dorronsoro et al. (Eds.): OLA 2021, CCIS 1443, pp. 254–266, 2021.
https://doi.org/10.1007/978-3-030-85672-4_19

characteristics making them hard to efficiently optimize. They are commonly solved by means of metaheuristics such as evolutionary algorithms or swarm intelligence [3]. However, the standard metaheuristics are not suitable to solve such large-scale global optimization (LSGO) problems because they suffer from the curse of dimensionality, i.e. their performance deteriorates when increasing the number of variables [1]. In this context, new approaches relying on the "divide-and-conquer strategy" have been proposed. They divide the initial LSGO problem into smaller ones which focus on smaller groups of variables. The latter are optimized in a round-robin fashion with a standard metaheuristic with the aim of producing the solution of the initial problem. This framework has been introduced by Potter and De Jong [9]. They designed a cooperative coevolutionary (CC) approach to optimize LGSO problems by means of a genetic algorithm. Following this promising approach, the CC strategy have been embedded in many other metaheuristics such as evolutionary programming [6], particle swarm optimization [2] and differential evolution [11].

In any case, the efficiency of this approach is highly dependent on the performed decomposition. The latter depends on the characteristics of the objective function in terms of separability. A function is separable if the influence of any variable on the function value depends only on itself [18]. In this case, any decomposition that reduces the dimensionality is efficient in the CC framework. Other functions can be classified as additively separable [8] if they can be written as:

$$f(x) = \sum_{i=1}^{m} f_i(x_i), \tag{1}$$

where x_i $(i = 1, \ldots, m)$ are mutually exclusive k_i-dimensional decision vectors of f_i, x is the n-dimensional decision vector of the function f and m is the number of independent components such that $k_1 + \cdots + k_m = n$. In this way, the influence, of any variable in a component, on the function value depends only on other variables of the same component. Therefore, an ideal decomposition would divide the initial problem such that each subproblem focuses on one component given in Equation (1). The main challenge is thus to identify these components. It can be done by using the differential grouping strategy [8,16].

However, separable and partially separable problems are not representative of most LSGO problems arising in real-world optimization applications. Most of them incorporate several components that usually interact with each other. For example, the supply chain design and optimization [4] involves several components such as suppliers, manufacturers and distributors that interact with each other through a variety of transportation and delivery methods. Such interconnected problems are often referred as overlapping problems [17] because they are composed of parts that overlap others. In other words, each component involves multiple variables and some of them are shared with one or several other components. This kind of function is very challenging and standard CC algorithms fail to optimize them efficiently. Indeed, most of them rely either on random grouping [18] or on intelligent decomposition methods based on interaction identification [8]. The former simply completes several random decompositions in order

to try catching linked variables in a same component but does not explicitly consider the interaction structure. The latter assigns all the linked variables in a single group and therefore does not reduce the dimensionality when dealing with overlapping problems. Two exceptions are the decomposition based on spectral clustering introduced in [5] and the decomposition specially designed for overlapping problems introduced in [15]. The latter breaks the linkage at shared variables between components in order to reduce the problem dimensionality, even for overlapping problems. It will be further discussed in Sect. 2.2.

In addition to the above methods, other CC strategies considering subsets that overlap each other have also received some attention. They raise some questions related to the exchange of information between components and related to the construction of the complete n-dimensional solution. In [14], non separable problems are decomposed into overlapping subproblems on the basis of a statistical variable interdependence learning scheme. The exchange of information is ensured by a periodically updated global solution (built on the basis of subproblem cores) used as shared memory. In [13], an overlapping decomposition covering the set of variables is predetermined. Compete and sharing strategies are implemented to choose the representative variables and share them among components. In [12], overlapping is not used to facilitate the decomposition but to overlap influential variables and evolves them in several components.

Some of these algorithms claim to tackle overlapping problems but do it with non-overlapped strategies [5,15]. Others, although based on overlapped strategies, do not explicitly claim to be able to tackle overlapping problems [12–14]. One may obviously think that the best way to optimize them in a CC framework is to do it with overlapped strategies. Nevertheless, to the best of the authors' knowledge, there are no research studies in that way. This paper introduces such a strategy and compare it with the non-overlapped approach specially designed for overlapping problems in [15]. The paper is organized as follows: Sect. 2 briefly describes the CC framework and the recursive differential grouping. Section 3 introduces the new strategy to split LSGO problems into overlapping subproblems and the overlapped CC framework that manages the exchange of information between subproblems. Experimental settings and results analysis are given in Sect. 4. Finally, findings and perspectives are discussed in Sect. 5.

2 Related Work

2.1 Cooperative Co-evolutionary Algorithms

The first attempt to optimize a LSGO problem with an evolutionary algorithm by means of a divide-and-conquer strategy was presented in 1994 [9]. Since then, this new approach, called cooperative co-evolution, has been widely studied [7]. The classical structure of this framework is described as follows:

1. *Decomposition*: Split the n-dimensional decision vector into some smaller disjoint subcomponents;

2. *Optimization*: Optimize each subcomponent with a standard evolutionary algorithm for a fixed number of iterations in a round-robin strategy;
3. *Combination*: Merge the solutions from each subcomponent to build the n-dimensional solution.

Throughout the optimization stage, the individuals in each subcomponent need to be evaluated with the n-dimensional function. For this purpose, they are completed with the variables of the *context vector*. The latter is a n-dimensional vector that contains information from all the subcomponents. Typically, it is composed of the variables of the current best solutions in each subcomponent and it is updated each time a better solution is found in a subcomponent.

2.2 Recursive Differential Grouping

In a CC framework, the decomposition should ideally be performed in such a way that there is no interaction between variables from different subcomponents. For additively separable problems, it can be uncovered with the Differential Grouping (DG) strategy [8,16]. In particular, the Recursive Differential Grouping (RDG) that benefits, as stated by its name, from recursive interaction detections between subsets of variables, relies on the following result [15,16]:

Theorem 1. *Let $f : \mathbb{R}^n \to \bar{\mathbb{R}}$ be an objective function; X_1 and X_2 be two mutually exclusive subsets of decision variables: $X_1 \cap X_2 = \emptyset$. If there exist a candidate solution $x^\star$ and sub-vectors a_1, a_2, b_1, b_2 such that*

$$f_{1,1}(x^\star) - f_{2,1}(x^\star) \neq f_{1,2}(x^\star) - f_{2,2}(x^\star) \tag{2}$$

where, $f_{i,j}(x^\star)$ is the function value obtained when replacing, in $x^\star$, the variables of X_1 with a_i and the variables of X_2 with b_j $(i, j = 1, 2)$, then there is some interaction between the decision variables in X_1 and X_2.

In practice, all the variables of $x^\star$, a_1 and b_1 are set to the lower bounds l of the search space. The variables of a_2 are set to the upper bounds u and those of b_2 are set to the mean $\bar{m}$ of the lower bounds and the upper bounds. Furthermore, equation (2) is not directly employed since the inequality may be the results of computational round-off errors instead of interaction detection, as expected. Thus, the following quantities are computed

$$\Delta_1 = f_{1,1}(x^\star) - f_{2,1}(x^\star),\ \Delta_2 = f_{1,2}(x^\star) - f_{2,2}(x^\star),\ \lambda = |\Delta_1 - \Delta_2| \tag{3}$$

and some interaction is detected when λ is greater than a threshold ϵ (see [16] for further details). Eventually, the success of the RDG algorithm relies on the recursive use of Theorem 1 to identify variables in X_2 that interact with those of X_1. Indeed, if any interaction between X_1 and X_2 is detected using Equation (3), the set X_2 is divided into two nearly equally-sized groups G_1 and G_2. Then, the interaction between X_1 and G_1 and X_2 and G_2 is checked. The process is repeated until all single variables in X_2 that interact with X_1 are identified.

In brief, the complete RDG algorithm can be presented as follows: (1) determine all the variables that interact with a selected variable x_i using the above recursive strategy and put them in a set X_1; (2) identify variables that interact with X_1 and add them to X_1, repeat the process until no more variable is added to X_1; (3) select another variable that is yet to be classified and return to step (1). Note that this approach would set all the variables of an overlapping problem into a single group. In [15], this issue was solved by slightly modifying the step (2) by imposing a condition on the size of X_1. In this new approach called RDG3, the step (2) is repeated: (a) until no more variable is added to X_1 or (b) until X_1 contains more than ϵ_n variables, where ϵ_n is fixed to a predetermined value.

3 Proposed Algorithm

The newly proposed algorithm aims to tackle LSGO overlapping problems within an overlapped CC framework. The fact that it has to deal with subcomponents that share several variables raises new challenges. The first one is to perform an accurate decomposition that detects overlapping variables efficiently and share them among several subcomponents. It can be achieved by using the modified approach of the RDG strategy presented in Sect. 3.1. The second challenge concerns the management of overlapping variables during the optimization, in particular for function evaluations. It will be discussed in Sect. 3.2.

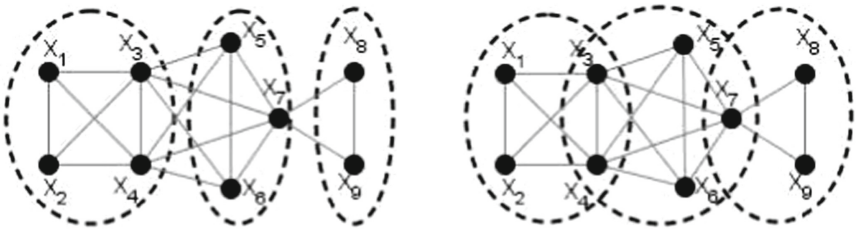
3.1 Overlapped Recursive Differential Grouping

The main idea of the newly proposed decomposition strategy is to relax the grouping by identifying variables that make the link between several components in interconnected problems and share them among these components. For example, in the interaction graph presented in Fig. 1, three components can be identified:

$$S_1 = \{x_1, x_2, x_3, x_4\}, S_2 = \{x_3, x_4, x_5, x_6, x_7\} \text{ and } S_3 = \{x_7, x_8, x_9\}. \quad (4)$$

In each of them, interaction between variables are plentiful while there is no direct interaction between variables from distinct components, i.e. $\forall\ i, j (i \neq j)$, $k, l (k \neq l)$such that $x_i \in S_k \backslash S_l$ and $x_j \in S_l \backslash S_k$, x_i does not interact with x_j. Using the RDG3 strategy to decompose such a problem will break the linkage at shared variables and will lead to the decomposition illustrated in Fig. 1a. The latter might not be the optimal one since x_3 and x_4 (resp. x_7) are not optimized with x_5, x_6 and x_7 (resp. x_8 and x_9) while they are strongly connected. The new strategy, called *Overlapped RDG* (ORDG), is aimed to allow some overlapping between subcomponents to prevent from breaking these important linkages. It will produce the decomposition proposed in Fig. 1b.

The ORDG strategy is presented in Algorithm 1. It is very closed to the RDG algorithm except for the instructions in the "else" statement at line 12. In particular, the instruction at line 5 recursively identifies variables in X_2 that interact with X_1. They are added to X_1 to constitute the set $X_1^\star$ (see Algorithm 2).



(a) RDG3 (b) Overlapped RDG

Fig. 1. The two obtained decompositions for an overlapping problem using RDG3 and Overlapped RDG strategies respectively.

Algorithm 1: Overlapped Recursive Differential Grouping

```
1  seps = {}, nonseps = {};
2  Set all the variables of x_l to the lower bounds, compute f = f(x_l) ;
3  X_1 = {x_1}, X_2 = {x_2, ..., x_n} ;
4  while X_2 ≠ {} do
5      X_1^⋆ =R_Inter(X_1, X_2, f, f) ;
6      if |X_1^⋆| = |X_1|
           // For RDG3, the if would be: if |X_1^⋆| = |X_1| or |X_1^⋆| > ε_n
7      then
8          if |X_1| = 1 then seps = seps ∪ X_1 ;
9          else nonseps = nonseps ∪ X_1 ;
10         X_1 = {x_j} s.t. j ≤ i ∀x_i ∈ X_2;
11         X_2 = X_2 \ {x_j} ;
12     else
           // For RDG3, the else statement would only contains the
              following instructions: X_1 = X_1^⋆,  X_2 = X_2 \ X_1 ;
13         if |X_1| = 1 then
14             X_1 = X_1^⋆,  X_2 = X_2 \ X_1 ;
15         else
16             X_1^⋆⋆ = L_inter(X_1, X_2, f, f) ;
17             nonseps = nonseps ∪ X_1 ;
18             X_1 = X_1^⋆ \ X_1 ∪ X_1^⋆⋆ ;
19             X_2 = X_2 \ X_1^⋆ ;
20 if |X_1| = 1 then seps = seps ∪ X_1 ;
21 else nonseps = nonseps ∪ X_1 ;
22 return seps and nonseps;
```

– If no interaction has been identified, i.e. if $|X_1^\star| = |X_1|$, the X_1 set is recognized as a nonseparable subset if it contains several variables, otherwise the only variable in X_1 is identified as a separable one (lines 8-9). The process moves on to the next variable that is yet to be classified (lines 10-11).

– Otherwise, some interaction has been identified between X_1 and X_2. The variables in X_2 responsible of the interaction have been identified during the recursive detection at line 5 but at this stage, the variables in X_1 responsible of the interaction have not yet been determined (and they should be to perform the overlapped decomposition). If X_1 contains only one variable, this is the one responsible of the interaction. In this case, the algorithm moves on to the next iteration while making the same update that for the RDG3 strategy (lines 13-14). Otherwise (i.e. if X_1 contains several variables), those interacting with X_2 are identified at line 16 using a recursive mechanism again (see Algorithm 3) and the update described in lines 17-19 produces the desired overlapped decomposition.

Algorithm 2: R_Inter$(X_1, X_2, \underline{f}, f)$

```
1 if Interact(X1, X2, f_, f) then
2     if |X2| = 1 then
3         X1 = X1 ∪ X2 ;
4     else
5         Split X2 into equally-sized
            groups G1, G2 ;
6         X1^1 =R_Inter(X1, G1, f_, f);
7         X1^2 =R_Inter(X1, G2, f_, f);
8         X1 = X1^1 ∪ X1^2 ;
9 return X1 ;
```

Algorithm 3: L_Inter$(X_1, X_2, \underline{f}, f)$

```
1 if Interact(X1, X2, f_, f) then
2     if |X1| = 1 then
3         return X1 ;
4     else
5         Split X1 into equally-sized
            groups G1, G2 ;
6         X1^1 =L_Inter(G1, X2, f_, f);
7         X1^2 =L_Inter(G2, X2, f_, f);
8         X1 = X1^1 ∪ X1^2 ;
9 return X1 ;
```

Note that the main difference between the `R_inter` and the `L_inter` functions lies in the fact that the former focuses on the set X_2 while the latter works on X_1. Furthermore, the two functions also differ in line 3 (see Algorithms 2 and 3) since the `R_inter` function adds variables from X_2 that interact with X_1 to X_1 while the `L_inter` function only returns variables from X_1 that interact with X_2. For both algorithms, the `Interact` function at line 1 relies on Theorem 1.

3.2 Overlapped CC Framework

The main layout of the overlapped CC framework is similar to the standard CC presented in Sect. 2.1 with the major exception that it is designed to detect and manage overlapping variables efficiently. For this purpose, the decomposition step performs the ORDG algorithm presented in the previous section.

The optimization still consists in iteratively evolving each subcomponent in a round-robin strategy. However, in this step, the cooperation between subproblems through the sharing of best solutions in the context vector needs to be revised. Since subcomponents overlap, a variable x_i belonging to one component S_k may also appear in another component S_l. This introduces the issue of which value of x_i has to be shared in the context vector. In a standard framework (i.e. without any overlapping), this value is the one of the variable x_i of the best

individual in the (only) subpopulation containing x_i (see Fig. 2a for an illustrative example). For the overlapped framework, this idea is extended: the value of x_i in the context vector is the one of the best individual among all the individuals in the two subpopulations focusing on x_i (or in the only subpopulation if x_i is not overlapped). Such an arrangement is illustrated in Fig. 2b.

Note that, in order to choose the best individual within two different subpopulations, the function value of each individual used for comparison is the one that has been computed during the optimization of the corresponding subcomponents in the round-robin fashion loop. In this process, individuals in each subcomponent are completed with the variables of the context vector in order to be evaluated. The latter is updated each time a better solution is reached.

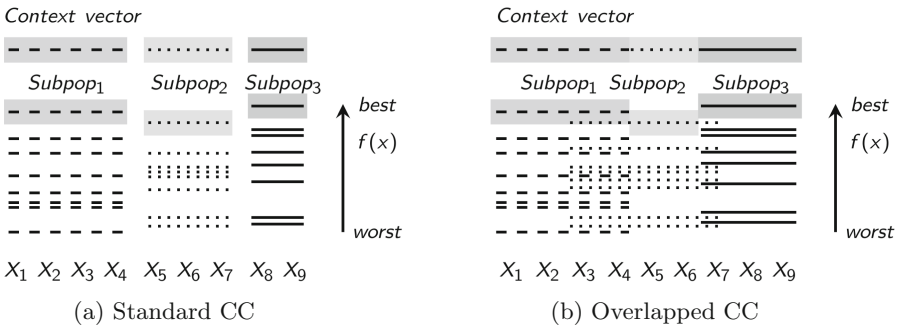


Fig. 2. Management of the context vector within a standard and an overlapped CC framework. The illustrative example relies on the interaction structure presented in Fig. 1. Dashed, dotted and solid lines represent individuals from subpopulations 1, 2 and 3 respectively. The context vector is built with the variables values of the best individual in each subpopulation.

4 Experimental Settings and Results

The performance of the new overlapped framework is assessed on large-scale overlapping benchmark problems derived from the CEC'2013 suite [17] and compared with the standard CC framework based on RDG3 decomposition [15]. The benchmark set contains 6 functions. Two of them, F_5 and F_6, are directly taken from [17]: F_6 is the 1000-d shifted Rosenbrock function and F_5 is the 905-d shifted Schwefel's function with conflicting[1] overlapping subcomponents. The four other functions, F_1 to F_4, are obtained by replacing the Schwefel basis function in F_5 by Ackley, Elliptic, Rastrigin and Rosenbrock functions respectively. Therefore, functions F_1 to F_5 contain 20 overlapping subcomponents that share 5 variables

[1] Note that the function f_{13} in [17] also contains overlapping subcomponents but it has not been included in the benchmark set because their overlapping subcomponents are conforming. It means that they have the same optimum value with respect to both subcomponent functions. It can be simply optimized in a standard CC framework.

with adjacent subcomponents. The F_6 function (Rosenbrock) can be seen as containing 999 subcomponents sharing one variable with adjacent ones.

In order to evaluate the decomposition effects of the newly proposed framework on overlapping problems, the RDG3 and ORDG strategies are used to decompose the benchmark problems presented above. For the RDG3, two different threshold values $\epsilon_n = 50$ and $\epsilon_n = 0$ are tested. The first value is the one used to study optimization results in [15] while the second value aims to identify as many components as possible and systematically cut the overlapping at shared variables. The number of components generated (k), the sum of the number of variables in each group (r) and the number of function evaluations computed (FEs) are reported in Table 1.

Table 1. Decomposition results of RDG3 (with $\epsilon_n = 50$ and $\epsilon_n = 0$) and ORDG strategies. k is the number of components generated, r is the sum of the number of variables in each group and FEs is the number of function evaluations.

Fun	RDG3 ($\epsilon_n = 50$)			RDG3 ($\epsilon_n = 0$)			ORDG		
	k	r	FEs	k	r	FEs	k	r	FEs
F_1	12	905	16273	20	905	16597	12	1011	16702
F_2	12	905	16252	19	905	16666	17	1000	18214
F_3	12	905	16249	20	905	16615	17	1000	18214
F_4	12	905	16252	20	905	16666	17	1000	18214
F_5	13	905	16288	21	905	16669	17	1003	18202
F_6	20	1000	49891	500	1000	25435	999	1998	59848

For the RDG3 decompositions, r is simply equal to the number of variables of the function because there is no overlap. For F_1 to F_5, the ORDG should capture the overlapping subcomponents introduced in [17] and therefore retrieve the 1000 variables involved in the benchmark construction. This is the case for F_2 to F_4. For F_1 and F_5, some additional interactions between independent variables have been identified due to computational round-off errors and lead to a slightly larger value of r. Still according to [17], the number of components k for functions F_1 to F_5 is equal to 20 in the benchmark construction. The RDG3 with $\epsilon_n = 50$ produces only 12 (or 13) components since components that contain less than 50 variables are merged with other ones. The RDG3 with $\epsilon_n = 0$ retrieves the 20 subcomponents (except for F_2 and F_5 for which the small difference is again due to computational round-off errors). The ORDG detects 17 components for functions F_2 to F_5[2]. They correspond to the ones formed in the benchmark construction except that some of them have been merged. Indeed, if the ORDG procedure starts the detection with a variable belonging to a component that share some overlapping variables with two adjacent components,

[2] Theoretically, 17 components should also be detected for F_1 but round-off erros affect the results for that particular function.

the latter are merged to form only one component. Thereafter, adjacent components to these components are also merged and so on. Although this prevents the detection of the 20 subcomponents, the obtained decomposition still agrees with the desired objective. In this particular case, the fact that some overlapping components contain two subsets of variables that do not directly interact will not affect the optimization efficiency. For the F_6 function, the obtained decomposition corresponds to the expected one, 20 (500) components of 50 (2) variables are formed for the RDG3 with $\epsilon_n = 50$ ($= 0$ resp.) and the ORDG produces 999 components of 2 variables. Finally, since the ORDG analyses additional interactions with respect to the RDG3, the cost in terms of FEs is higher. However, the additional cost remains reasonable and will be negligible with respect to the budget in terms of FEs allowed for the optimization.

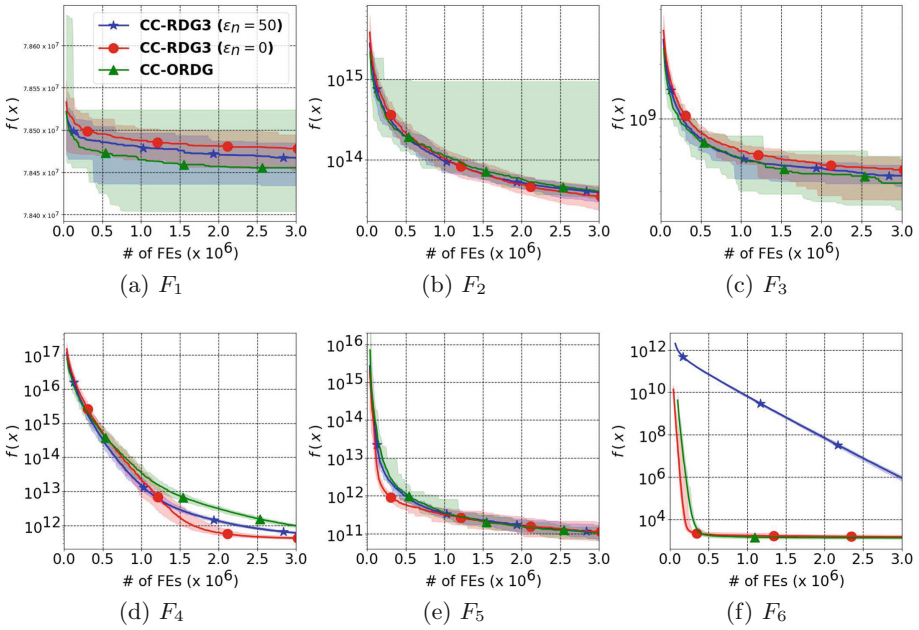
The influence of the decomposition on the optimization results is analyzed by embedding each kind of decomposition in the overlapped CC framework presented above. In particular, when the latter is coupled with the RDG3 decompositions, it behaves like the standard CC. The evolutionary algorithm used to optimize the subcomponents is a genetic algorithm. In this study, the one implemented in the Minamo software is considered [10]. Here there is an overview of its main features: real-value representation of the individuals; tournament selection to pick up pairs of parents; arithmetic crossovers for recombination; mutation rate of 1 %; elitism of two individuals. Within the CC framework, the population size is set to 10 times the number of variables of the considered component. The round-robin fashion optimization loop is repeated until the maximum number of FEs is reached. It is set to 3×10^6 in total (for the decomposition and the optimization).

The median of the best solution over 51 independent runs and the standard deviation are reported in Table 2. The CC-ORDG produces better solution quality than the CC-RDG3 for 4 of the 6 functions. The CC-RDG3 with $\epsilon_n = 0$ generates the best results for the 2 other functions. Convergence graphs depicting the convergence behavior along the optimization process are also provided in Fig. 3. It can be seen that the three algorithms follow the same trend for functions F_1 to F_5[3]. Between the two variants of the CC-RDG3, the slightly different number of components does not have too much influence on the optimization quality. However, for F_6, the CC-RDG3 with $\epsilon_n = 0$ that produces many more subcomponents (each of them focusing on 2 variables) has a better handle of the optimization. Furthermore, the closed results between the CC-RDG3 with $\epsilon_n = 0$ and the CC-ORDG may be surprising. By analyzing the convergence behavior of the overlapping variables in the CC-ORDG in details, it can be seen that most of the time, the variables shared among two subcomponents converge to the same value at the same rate in the two subcomponents. In this context, the overlapped decomposition does not significantly contribute to a better cooperation between subcomponents in comparison with the cooperation through the sharing of the context vector performed in a standard CC framework. Therefore,

[3] Note that for F_2, the CC-ORDG is stuck in a pseudo-optima for a few runs. It causes the large green-colored area in Fig. 3b.

Table 2. Optimization results of the CC-RDG3 (with $\epsilon_n = 50$ and $\epsilon_n = 0$) and the CC-ORDG. The median of the best solution over 51 independent runs and the standard deviation are presented. Best median values are in bold.

Fun	RDG3 ($\epsilon_n = 50$)		RDG3 ($\epsilon_n = 0$)		ORDG	
	Median	Std	Median	Std	Median	Std
F_1	7.03e+07	1.77e+05	7.04e+07	1.48e+05	**7.01e+07**	2.84e+05
F_2	3.95e+13	3.56e+12	**3.53e+13**	5.24e+12	3.85e+13	1.67e+14
F_3	4.83e+08	2.81e+07	5.22e+08	4.45e+07	**4.17e+08**	8.11e+07
F_4	6.01e+11	1.82e+10	**4.27e+11**	1.40e+10	7.80e+11	3.58e+10
F_5	1.04e+11	2.15e+10	1.15e+11	2.66e+10	**9.64e+10**	1.75e+10
F_6	8.68e+05	9.80e+04	1.50e+03	1.21e+02	**1.34e+03**	1.01e+02



(a) F_1 (b) F_2 (c) F_3

(d) F_4 (e) F_5 (f) F_6

Fig. 3. Convergence graphs representing the evolution of $f(x)$ (in log-scale) with respect to the number of FEs. CC-RDG3 with $\epsilon_n = 50$ (blue stars), CC-RDG3 with $\epsilon_n = 0$ (red circles), CC-ORDG (green triangles). The solid line depicts the median value while the light-colored area represents the interval between the best and the worst value over the 51 runs.

although results in Table 2 might indicate that the CC-ORDG provides slightly better results, we can not definitely claim that a strategy is better than the other.

5 Discussion

The new CC framework introduced in this paper is designed to optimize overlapping LSGO problems with an overlapped decomposition strategy. In this context, an overlapped variant of the RDG has been developed to efficiently detect overlapping variables and share them among several subcomponents. The optimization step of the standard CC framework has also been extended in order to efficiently sha-re information between overlapped subcomponents through the context vector.

Numerical experiments were conducted on 6 benchmark functions. The extension of the method to a larger set of test functions is straightforward. However we believe the latter goes beyond the scope of this introductory paper and thus it will be considered in a further work. Similarly, the benchmark set is limited to 905-d and 1000-d functions, which is common practice in LSGO studies. Further research on the scalability may also be carried out to determine how the algorithm performs on more complex problems with larger dimensions.

The experiments presented in this paper show that the new approach produces the desired overlapped decomposition. However, although the optimization results might indicate that the new decomposition helps to get slightly better solutions, we can not definitely claim that the new framework outperforms the standard ones. This may be partly explained by the fact that the exchange of information between subcomponents in a standard CC framework through the context vector is stronger than we could expect. In any way, there is scope for even better progress to further develop the CC concept to deal with overlapping problems. We think that the new strategy that introduces overlapped subcomponents may be a promising way to achieve such an improvement.

References

1. Bellman, R.: Adaptive Control Processes : A Guided Tour. Princeton University Press, Princeton, New Jersey, USA (1961)
2. van den Bergh, F., Engelbrecht, A.P.: A cooperative approach to particle swarm optimization. IEEE Trans. Evol. Comput. **8**, 225–239 (2004)
3. Eiben, A., Smith, J.: Introduction to Evolutionary Computing. Natural Computing Series. Springer, Heidelberg, Germany (2015)
4. Garcia, D.J., You, F.: Supply chain design and optimization: challenges and opportunities. Comput. Chem. Eng. **81**, 153–170 (2015)
5. Li, L., Fang, W., Wang, Q., Sun, J.: Differential grouping with spectral clustering for large scale global optimization. In: 2019 IEEE Congress on Evolutionary Computation, pp. 334–341. IEEE, Piscataway, New Jersey, USA (2019)
6. Liu, Y., Yao, X., Zhao, Q., Higuchi, T.: Scaling up fast evolutionary programming with cooperative coevolution. In: Proceedings of the 2001 Congress on Evolutionary Computation, Vol. 2, pp. 1101–1108. IEEE, Piscataway, New Jersey, USA (2001)
7. Mahdavi, S., Shiri, M.E., Rahnamayan, S.: Metaheuristics in large-scale global continues optimization: a survey. Inf. Sci. **295**, 407–428 (2015)
8. Omidvar, M.N., Li, X., Mei, Y., Yao, X.: Cooperative co-evolution with differential grouping for large scale optimization. IEEE Trans. Evol. Comput. **10**(10), 1–17 (2013)

9. Potter, M.A., De Jong, K.A.: A cooperative coevolutionary approach to function optimization. In: Davidor, Y., Schwefel, H.P., Männer, R. (eds) Parallel Problem Solving from Nature - PPSN III, **866**, 249–257. Lecture Notes in Computer Science. LNCS. Springer, Heidelberg, Germany (1994). https://doi.org/10.1007/11539117_147
10. Sainvitu, C., Iliopoulou, V., Lepot, I.: Global optimization with expensive functions - sample turbomachinery design application. In: Diehl, M., et al. (eds.) Recent Advances in Optimization and its Applications in Engineering, 499–509. Springer-Verlag, Berlin, Heidelberg (2010). https://doi.org/10.1007/978-3-642-12598-0_44
11. Shi, Y., Teng, H., Li, Z.: Cooperative co-evolutionary differential evolution for function optimization. In: wang, L., Chen, K., Ong, Y.S. (eds) Advances in Natural Computation. ICNC 2005, **3611**, 1080–1088. Lecture Notes in Computer Science. LNCS, Springer, Heidelberg, Germany (2005). https://doi.org/10.1007/11539117_147
12. Song, A., Chen, W.N., Luo, P.T., Gong, Y.J., Zhang, J.: Overlapped cooperative co-evolution for large scale optimization. In: 2017 IEEE International Conference on Systems, Man, and Cybernetics, pp. 3689–3694. IEEE, Piscataway, New Jersey, USA (2017)
13. Strasser, S., Sheppard, J., Fortier, N., Goodman, R.: Factored evolutionary algorithms. IEEE Trans. Evol. Comput. **2**(21), 281–293 (2017)
14. Sun, L.S.Y., Cheng, X., Liang, Y.: A cooperative particle swarm optimizer with statistical variable interdependence learning. Inf. Sci. **1**(186), 20–39 (2012)
15. Sun, Y., Li, X., Ernst, A., Omidvar, M.N.: Decomposition for large-scale optimization problems with overlapping components. In: 2019 IEEE Congress on Evolutionary Computation, pp. 326–333. IEEE, Piscataway, New Jersey, USA (2019)
16. Sun, Y., Omidvar, M.N., Kirley, M., Li, X.: Adaptive threshold parameter estimation with recursive differential grouping for problem decomposition. In: Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2018, pp. 889–896. ACM, New York, NY, USA (2018)
17. Xi, L., Tang, K., Omidvar, M.N., Yang, Z., Qin, K.: Benchmark functions for the CEC 2013 special session and competition on large-scale global optimization. Technical report. RMIT University, Melbourne (2013)
18. Yang, Z., Tang, K., Yao, X.: Large scale evolutionary optimization using cooperative coevolution. Inf. Sci. **178**(15), 2985–2999 (2008)